



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA3712

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Code of Conduct:

*I **KAVYA SHRI G (192421052)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

G. Kavya Shri



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

- 1. A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.**

Testing as an engineering activity means replacing random checks with a planned, systematic, measurable, and repeatable process. For a banking application, the test engineer should first review requirements and identify high-risk areas such as fund transfers, interest calculations, authentication, and account balances. Each requirement should have clear acceptance criteria and be linked to test cases through a requirements traceability matrix. A test plan should define responsibilities, environments, test data, schedules, and release conditions. Testing should begin during requirement analysis so that ambiguous rules and design defects are discovered before they become costly implementation failures.

Systematic test-design techniques should replace reliance on ad-hoc testing. Equivalence partitioning divides inputs into valid and invalid groups, while boundary value analysis checks values near transaction limits. Decision-table testing checks combinations of conditions such as sufficient balance, valid authorisation, and daily transfer allowance. State-transition testing verifies transaction movement between pending, completed, failed, and reversed states. Unit, integration, system, and acceptance testing should provide complementary coverage. Security, performance, concurrency, and recovery testing are particularly important because a banking feature must remain secure and financially correct even during heavy traffic or component failures. Structured exploratory testing can supplement these techniques to uncover unexpected problems.

Tools make this process consistent and efficient. JUnit or pytest can automate unit tests, REST Assured can test banking APIs, and Selenium or Playwright can verify critical customer journeys. JMeter can assess performance, while security tools such as OWASP ZAP can support vulnerability detection. A defect-management tool should record severity, reproduction steps, and evidence, and a continuous-integration pipeline should run automated regression tests whenever code changes. ISO/IEC/IEEE 29119 can guide test processes and documentation, while OWASP ASVS can support security verification. The team should use controlled environments and synthetic or masked customer data, with release gates requiring successful critical tests and no unresolved release-blocking defects.

For example, assume a bank permits transfers from ₹1 to ₹50,000, in increments of ₹0.01, with sufficient available balance and valid authorisation. An ad-hoc tester might successfully transfer ₹1,000 and overlook important failures. Boundary testing would instead check ₹0.99, ₹1, ₹1.01, ₹49,999.99, ₹50,000, and ₹50,000.01. Decision-table tests would verify that transfers are rejected when funds or authorisation are insufficient. A fault-injection test would interrupt processing between debit and credit to verify atomicity: either both changes become committed or neither does. Retrying a request with the same idempotency key would verify that money is transferred only once, and simultaneous-transfer tests would check that concurrent requests cannot spend the same available funds twice.

This approach improves defect detection by targeting boundaries, business-rule combinations, and failure conditions that random testing often misses. For instance, it can expose an incorrect comparison that rejects exactly ₹50,000 or a retry defect that produces duplicate debits. Once corrected, these scenarios become automated regression tests that guard against recurrence. Tracking escaped defects, critical-risk coverage, and recurring failure causes provides evidence of improvement. Thus, testing becomes an engineering discipline that reduces financial risk and builds justified confidence in the application's reliability, security, and correctness.

2. **An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.**

ATM testing should be integrated into every phase of the Software Development Life Cycle (SDLC), not performed only after coding. I would establish documented procedures, traceable test cases, controlled environments, and clear entry and exit criteria. The main quality objective is to ensure that only authorised withdrawals succeed, the correct cash is dispensed, and account records remain accurate when failures occur.

Requirements Analysis- Review withdrawal requirements with banking stakeholders and convert them into testable acceptance criteria. These should define PIN verification, available balance, withdrawal limits, supported denominations, ATM cash availability, and interrupted-transaction handling. For example, withdrawals must be positive multiples of ₹100 and limited to ₹20,000 per transaction. Requirements should also specify daily limits and how failed or uncertain cash dispensing is reconciled.

Test Planning- A risk-based test plan would define scope, responsibilities, resources, schedules, test levels, and defect procedures. Incorrect debits, unauthorised withdrawals, and cash-dispensing failures would receive the highest priority. The test environment would include an ATM simulator, test banking host, synthetic accounts, and representative dispenser hardware. Exit criteria would require all critical withdrawal tests to pass and no unresolved release-blocking defects.

System and Test Design- Design reviews would examine interactions between the ATM, authentication service, banking host, and dispenser, especially timeout and recovery behaviour. Tests would use equivalence partitioning, boundary value analysis, decision tables, and state-transition testing. Under the assumed rules, ₹0 and ₹150 should be rejected, while ₹100 and ₹20,000 should be accepted when other conditions are satisfied; ₹20,100 should be rejected.

Implementation and Unit Testing- Automated unit tests would cover amount validation, balance checks, limit calculations, and transaction-state handling. Code reviews and static analysis would help identify logic and security defects early. Continuous integration would provide rapid feedback and prevent regression.

Integration Testing- Integration tests would verify communication between the ATM, banking host, and dispenser using normal and failure responses. Scenarios would include declined authorisation, network timeouts, duplicate messages, dispenser errors, and lost acknowledgements. If cash is dispensed but the completion message is lost, recovery must prevent a second withdrawal or incorrect reversal. Confirmed dispensing failures must resolve any debit or hold according to approved recovery rules.

System and Acceptance Testing- Complete withdrawal journeys would be tested under realistic security, performance, usability, concurrency, and recovery conditions. An insufficient-funds attempt should dispense no cash and create no debit. Banking representatives would validate workflows, messages, receipts, and recovery procedures.

Deployment and Maintenance- Before release, I would execute regression tests, verify configuration, and perform controlled deployment checks. After deployment, the team would monitor failed withdrawals, reconciliation discrepancies, and recovery outcomes without exposing customer data. Each production defect would undergo root-cause analysis and become a regression test where appropriate. For example, a duplicate debit caused by a retry would lead to permanent duplicate-message tests.

A consistent process reduces defects through reviewed requirements, standard test methods, version-controlled tests, reproducible data, and release gates. Success should be measured through escaped-defect rates, critical-requirement coverage, reconciliation discrepancies, and recovery times—not test pass rates alone.

- 3. A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.**

When a web application shows a mismatch between expected and actual results during login or transaction operations, an error is a human mistake made during requirements, design, coding, configuration, or testing. For example, a developer may misunderstand the login rule and implement account lockout after three failed attempts instead of five. This mistake creates a defect, which is a flaw in the software, such as code that locks the account after three attempts. When the application runs and locks a user after the third incorrect password, the visible incorrect behaviour is called a failure. Thus, the usual relationship is error → defect → failure, although a defect may not always cause a failure if the affected code is never executed.

For example, during login, a developer may accidentally reverse the password-comparison condition. The resulting defect may reject valid credentials or accept invalid ones, causing a failure when a genuine user cannot log in or an unauthorised user gains access. During a transaction, a developer may fail to consider a repeated request after a network timeout. This creates a defect in which the system processes the same request twice. The resulting failure could be a ₹5,000 payment being processed twice and the customer being debited ₹10,000 instead of ₹5,000. Similarly, an incorrect balance-check condition may allow a transaction despite insufficient funds.

Misunderstanding these terms can reduce testing effectiveness because testers may report only the visible symptom, developers may fix the wrong component, duplicate failures may be counted as separate defects, and management may receive inaccurate defect statistics. For instance, reporting payment failed does not explain whether the cause was invalid input, a timeout, incorrect balance validation, or duplicate processing. Defect reports should therefore clearly state the title, environment, application version, preconditions, test data, exact reproduction steps, expected result, actual result, severity, priority, reproducibility, and supporting evidence such as sanitised logs or screenshots. A clear report could state: After a ₹5,000 payment times out and the same request is retried, the account is debited twice; the expected result is one debit only. Such precise reporting helps developers reproduce the failure, identify the underlying defect, determine its root error, and implement an effective correction.

- 4. A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.**

When most defects are concentrated in a few modules, the team should apply software testing principles to focus effort where the risk is highest. Defect clustering states that a small number of modules usually contain most of the defects. Therefore, testers should analyse previous defect data, code complexity, recent changes, failure history, and business impact to identify high-risk modules. These modules should receive more test cases, deeper exploratory testing, automation, code reviews, and regression coverage, while stable low-risk modules can receive proportionate testing. This follows the risk-based testing principle and avoids wasting equal effort on every part of the system.

Other testing principles also guide test planning. Since exhaustive testing is impossible, the team should select representative and high-value scenarios instead of attempting every input combination. The early testing principle requires reviewing requirements, designs, and interfaces before implementation so that defects are detected when they are cheaper to fix. The pesticide paradox indicates that repeatedly running the same tests eventually finds fewer new defects; therefore, test data, techniques, and scenarios should be regularly updated. The team should also remember that testing shows the presence of defects, not their complete absence, and that a system may still be unacceptable if it does not meet business requirements.

Test cases can be prioritized using risk, business criticality, defect history, frequency of use, and failure impact. For example, in an online banking application, a payment-processing module with many previous defects should be tested before a rarely used profile-editing module. Within payment testing, cases for valid payment, insufficient balance, duplicate submission, timeout recovery, incorrect debit, refund, and security validation should receive the highest priority. A simple priority order could be: P1—critical transactions and security failures; P2—important business workflows and boundary conditions; P3—minor usability and cosmetic issues. Each requirement should be linked to test cases through a traceability matrix, and automated smoke and regression tests should run first in continuous integration.

A real-time example is an e-commerce website where most production defects occur in the checkout and payment modules. The team can analyse incident reports and discover that failures are concentrated around discount calculation, payment timeouts, and order creation. Test planning would therefore give these areas additional boundary-value, integration, concurrency, and recovery tests. High-priority tests would verify that an order is created only once, the correct amount is charged, discounts are applied correctly, and a failed payment does not create an unpaid order. Lower-risk tests, such as changing a profile picture, could be scheduled later. Applying these principles improves efficiency by detecting serious defects earlier, reducing duplicate effort, improving regression coverage, shortening investigation time, and ensuring that limited testing resources are directed toward areas that pose the greatest risk to users and the business.

5. **A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.**

A defect repository should not be used only to record defects. The team should also study it to prevent the same problems from appearing again.

Defects can begin at different stages:

- Requirements: The requirement is unclear or misunderstood.
- Design: The system design does not handle a situation correctly.
- Coding: The program contains an error.
- Integration: Different modules do not work correctly together.
- Environment: Problems occur because of hardware, software, or configuration.

Defects can also be grouped by type, such as functional, performance, security, usability, compatibility, or data-related defects. Each defect should also have a severity and priority level.

A useful defect record should include:

- Steps to reproduce the defect
- Expected and actual results
- Affected module
- Severity and priority
- Root cause
- Release in which it was found
- Fix details
- Related test cases

Repository data can help testers improve test planning. For example, if the repository shows that checkout defects occur repeatedly, testers should create more tests for payment, discounts, duplicate orders, refunds, and timeout situations. These tests should be added to the regression suite so that they run after every future change.

Suppose an online shopping application repeatedly has problems with discount calculations and duplicate orders. The team can use old defect records to create tests for expired coupons, invalid coupons, different purchase amounts, repeated payment attempts, and network failures. These tests can also be automated.

To prevent recurring defects, the company should:

- Find the root cause instead of fixing only the symptom.
- Add a regression test for every fixed defect.
- Improve unclear requirements.
- Conduct design and code reviews.
- Use common defect categories and proper documentation.
- Analyse defect trends after every release.
- Share defect information with developers and testers.

Using the repository in this way will reduce repeated defects, improve test coverage, lower maintenance effort, and make future releases more stable and reliable.

- 6. A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.**

A mobile banking application that frequently crashes during fund transfers should be tested systematically to identify the exact cause of the problem. The tester should first reproduce the crash and record details such as the device, operating system, application version, network condition, transfer amount, beneficiary details, and the step at which the crash occurs. Crash logs, error messages, API responses, and server records should then be analyzed. Possible causes may include invalid input, insufficient balance, expired sessions, weak network connectivity, unexpected server responses, or repeated tapping of the transfer button.

Testing should cover both normal and abnormal conditions, such as successful transfers, insufficient balance, incorrect OTP, daily transfer limits, invalid beneficiaries, network failure, and closing the application during a transfer. For example, if the server returns a PENDING status but the application handles only SUCCESS and FAILED, it may crash while processing the response. This shows that the application does not properly handle all transaction states.

Different types of testing can help prevent such failures. Unit testing can check validation and transaction logic, while integration and API testing can verify communication with banking servers. System testing can test the complete transfer process, and regression testing can ensure that fixes do not affect existing features. Security, performance, compatibility, and recovery testing are also important.

Future failures can be reduced by using automated tests, code reviews, crash-monitoring tools, and simulated network failures. The transfer button should be disabled after one click, and unique transaction IDs should be used to prevent duplicate transfers. If a network failure occurs, the application should check the transaction status before allowing a retry. Through proper error handling, continuous testing, and careful monitoring, the application can become more stable, secure, and reliable.

7. A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.

Software testing and debugging are related but different activities. Testing is the process of executing software to find failures and determine whether it satisfies specified requirements. Testers usually identify defects by using test cases, test data, expected results, and different testing techniques. Debugging, on the other hand, is a development activity performed after a failure or defect has been reported. It involves locating the source of the problem, analyzing the code, correcting it, and verifying that the correction works.

Testing contributes to software quality by detecting incorrect behavior, missing requirements, security weaknesses, performance problems, and compatibility issues. Debugging contributes by removing the causes of those failures. For example, testing may show that a banking application crashes when a transfer request receives a PENDING response. Debugging then examines the logs, API response handling, and program code to find why the application cannot process that status.

Confusing testing with debugging can lead to ineffective defect resolution. If developers repeatedly change code without first reproducing the failure or understanding its conditions, they may fix only the visible symptom. Similarly, if testers attempt to correct the code instead of reporting clear evidence, important information such as steps to reproduce, test data, logs, and expected results may be lost. For example, a tester may report only transfer failed, while the actual problem occurs only when the network disconnects after the transaction is submitted. Without proper testing details, the developer may not identify the real cause, and the defect may reappear.

A structured approach should begin with testing. The tester should execute the test case, compare the actual result with the expected result, and record the failure with complete evidence. The developer should then reproduce the issue, analyze logs and code, identify the root cause, implement a fix, and perform unit testing. After that, the tester should conduct retesting to confirm that the defect is fixed and regression testing to ensure that other features have not been affected.

Effective integration also requires clear communication and defined responsibilities. Testers should report defects using a standard format containing the environment, preconditions, steps, expected result, actual result, severity, and supporting evidence. Developers should document the root cause and the changes made. Automated tests and continuous integration can further ensure that every fix is checked regularly.

Therefore, testing answers:

What failed and under which conditions?, while debugging answers Why did it fail and how can it be corrected?

Using both processes in the correct order helps the team detect defects accurately, remove their root causes, and produce more reliable software.

8. An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.

An e-commerce platform may behave differently across browsers because browsers use different rendering engines, JavaScript versions, CSS rules, security policies, and storage mechanisms. Differences may appear in product display, search, login, shopping-cart updates, payment processing, or responsive layouts. Therefore, cross-browser testing should be planned according to business risk and user popularity rather than testing every possible browser combination.

The testing strategy should begin by identifying the supported browsers, operating systems, devices, screen sizes, and browser versions. Important user journeys such as registration, login, product search, filtering, adding items to the cart, applying coupons, checkout, payment, and order confirmation should be tested on each major browser. Both normal and abnormal conditions should be included, such as slow networks, disabled JavaScript, expired sessions, invalid coupons, empty carts, and interrupted payments. The same test case should be executed in every selected environment, with the expected result clearly defined.

Defects may originate when developers use browser-specific CSS properties, unsupported JavaScript functions, incorrect HTML, or features that behave differently between browsers. For example, a checkout button may work in Chrome but remain invisible in Safari because of an unsupported CSS rule. Similarly, a date or currency value may be processed differently by different browsers, causing incorrect order totals. Caching, cookies, local storage, pop-up blocking, font rendering, and security restrictions can also create inconsistent behavior.

Systematic testing improves reliability by detecting these differences before release. Testers should record the browser, version, operating system, device, steps to reproduce, screenshots, console errors, and actual and expected results. Defects should be fixed and then verified through retesting and regression testing across all supported environments. Automated tests can repeatedly check essential functions, while visual comparison can identify layout and styling differences.

Useful tools and techniques include Selenium, Cypress, Playwright, BrowserStack, Sauce Labs, and LambdaTest for browser automation and real-device testing. Responsive testing can be performed using browser developer tools and mobile emulators. Test cases should use equivalence partitioning and boundary-value analysis for inputs such as prices, quantities, discounts, and addresses. Compatibility matrices, risk-based testing, exploratory testing, automated regression suites, and continuous integration can provide better coverage. A staged release and real-user monitoring can also reveal browser-specific problems that were missed during testing.

9. A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.

A payroll system may produce incorrect salary calculations because of logical defects in formulas, conditions, or data processing. These defects can be classified as calculation defects, such as incorrect basic salary or overtime formulas; boundary-value defects, such as incorrect handling of tax limits or overtime hours; business-rule defects, such as applying the wrong deduction or bonus rule; data-handling defects, such as using outdated employee information; and conditional or control-flow defects, such as applying a deduction even when an employee is not eligible.

Understanding defect classes helps testers design more effective test cases. For calculation defects, testers should compare system results with manually calculated salaries. Boundary testing should be performed around tax slabs, minimum wages, overtime limits, and bonus thresholds. Business-rule testing should verify deductions, allowances, leave payments, and employee categories. For example, if tax is applied only when salary exceeds ₹50,000, tests should include salaries just below, exactly at, and just above ₹50,000. This can reveal incorrect comparison operators such as < being used instead of <=.

These defects can be prevented by clearly documenting payroll rules, reviewing formulas, using modular code, and applying unit testing to each salary component. Test data should include different employee types, salaries, working hours, deductions, bonuses, and tax conditions. Automated regression tests should run whenever payroll logic is changed. Code reviews and static analysis can also identify incorrect conditions and unused or wrongly calculated variables.

A better testing strategy should include unit, integration, system, regression, boundary-value, equivalence-partitioning, and data-validation testing. Testers should maintain a traceability matrix linking payroll requirements to test cases and should verify both individual salary components and the final net salary. Results should also be checked against an independent payroll calculator or spreadsheet. Proper defect classification, strong test coverage, automation, and regular review of business rules can significantly reduce salary-calculation errors in future releases.

10. A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.

Testing should be treated as an engineering activity rather than a random process. The team should begin by understanding the requirements, identifying important features, defining expected results, and preparing test cases before execution. Each test case should include a unique ID, objective, preconditions, input data, execution steps, expected result, and actual result. This creates a clear and repeatable testing process.

Structured test case design is important because random testing may miss critical conditions and produce inconsistent results. For example, in an online shopping system, randomly testing the checkout page may not reveal an error in discount calculations. A properly designed test set would include valid coupons, expired coupons, invalid coupons, minimum purchase limits, and discounts at boundary values. This increases the chance of detecting both common and hidden defects.

A systematic process should include requirement analysis, test planning, test design, test execution, defect reporting, retesting, and regression testing. Testers should use techniques such as equivalence partitioning, boundary-value analysis, decision-table testing, state-transition testing, and error guessing. For instance, if a system accepts ages from 18 to 60, boundary testing should check 17, 18, 60, and 61 instead of testing only a random age.

Suitable techniques to improve test effectiveness include:

Equivalence partitioning -> Reduces test cases by grouping similar inputs.

Boundary-value analysis -> Finds errors at limits and threshold values.

Decision-table testing -> Tests combinations of business conditions.

State-transition testing -> Verifies changes between system states.

Risk-based testing -> Focuses on critical and high-impact features.

Exploratory testing -> Helps discover unexpected defects.

Automation -> Speeds up repeated regression testing.

Traceability matrix -> Ensures every requirement is tested.

Using well-designed test cases improves defect detection because tests become complete, measurable, repeatable, and easier to review. It also helps the team identify untested requirements, reproduce defects accurately, and confirm that fixes have not introduced new problems. Thus, structured testing provides better coverage and produces more reliable software than random test execution.

11. A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.

A healthcare application showing inconsistent data across modules should be tested systematically because inaccurate patient information can create serious risks. Testers should verify whether data entered in one module, such as patient registration, is correctly displayed in other modules such as appointments, billing, laboratory results, and medical records. Testing should include creating, updating, and deleting patient information, checking different user roles, and verifying data after synchronization, network failure, or session timeout.

Defects may originate from incorrect database mapping, inconsistent field names, failed API communication, outdated cached data, improper validation, or poor transaction handling. For example, a patient's updated phone number may appear correctly in the registration module but remain unchanged in the billing module because the update was not synchronized properly. Differences in data formats, database queries, or access permissions can also cause inconsistent results.

Systematic testing ensures data integrity by comparing the source data with the information displayed in every connected module. Integration and end-to-end testing should verify that data is transferred correctly, while database testing can confirm that records are stored without duplication or loss. Boundary testing, negative testing, concurrency testing, and recovery testing should also be performed. Automated tests can repeatedly compare records across modules and detect inconsistencies early.

Defect tracking should use a standard reporting format containing the defect title, patient-data scenario, environment, steps to reproduce, expected result, actual result, severity, screenshots, logs, and affected modules. Each defect should be assigned to the responsible team and tracked through statuses such as New, Assigned, Fixed, Retest, and Closed. Defects involving incorrect medical data should receive high priority. Clear reports, unique defect IDs, root-cause analysis, regular reviews, and regression testing after every fix can improve communication and prevent the same data-integrity problem from recurring.

12. A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.

A defect repository is a central database that stores information about defects found during development and after release. It may contain the defect description, module, severity, priority, environment, root cause, developer responsible, resolution, and date of detection. If a company continues to face repeated defects despite testing, the repository should be analyzed instead of treating each defect as an isolated problem.

The repository helps testers design better test cases by showing which modules fail frequently, which defect types are most common, and which conditions usually cause failures. For example, if many defects are related to incorrect input validation, testers should create more boundary-value and negative test cases. If defects are repeatedly found during integration, stronger API and end-to-end testing should be included.

Repository information : How it supports testing

Frequently affected module -> Gives priority to high-risk areas.

Defect severity -> Helps focus on business-critical failures.

Root cause -> Guides the selection of suitable test techniques.

Defect-reopening rate -> Shows whether fixes are incomplete.

Environment or browser details -> Supports compatibility testing.

Time of detection -> Identifies whether defects escape to later stages.

Defect patterns can be identified by grouping repository records according to module, defect type, release, severity, developer team, and testing phase. For example, analysis may show that most defects occur in the payment module, appear after database changes, or are discovered only during production testing. This indicates weaknesses in payment validation, database integration, or pre-release testing. Repeated defects with similar causes may also indicate unclear requirements, insufficient code reviews, or incomplete regression tests.

To prevent recurring defects, the organization should perform root-cause analysis for high-severity and repeated issues. Corrective actions may include improving requirements, adding missing automated tests, strengthening code reviews, updating coding standards, and creating regression tests for every confirmed defect. The repository should also be reviewed after each release to identify trends and measure improvement.

A better testing strategy should combine risk-based testing, unit testing, integration testing, system testing, regression testing, and exploratory testing. Defect metrics such as defect density, defect leakage, reopening rate, and average resolution time should be monitored. Linking requirements, test cases, defects, and code changes will improve traceability. In this way, the defect repository becomes a learning tool that supports test planning, reveals weak areas, and helps prevent the same defects from appearing in future releases.

13. A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.

A web-based learning system that fails during peak usage indicates that the testing process did not adequately cover performance, scalability, and high-concurrency scenarios. The development team may have tested the system with only a small number of users under normal conditions, while ignoring situations such as thousands of students logging in simultaneously, watching videos, downloading study materials, submitting assignments, or taking online examinations at the same time. Proper test planning is important because it defines the expected workload, number of concurrent users, test environment, performance objectives, response-time limits, acceptable error rate, and recovery requirements. Test coverage should include normal-load testing, peak-load testing, stress testing, spike testing, endurance testing, and recovery testing. For example, the system should be tested when the number of users suddenly increases, when large files are uploaded by many students, and when the database or network becomes temporarily unavailable. If these scenarios are not included, serious defects may remain hidden until the system is used in real conditions. Systematic testing improves reliability by using realistic user behavior and production-like data to identify bottlenecks in application code, databases, servers, networks, and third-party services. Tools such as Apache JMeter, LoadRunner, Gatling, and k6 can simulate large numbers of users, while monitoring tools such as Grafana, Prometheus, and Application Insights can measure CPU usage, memory consumption, database connections, response time, throughput, and failure rates. After correcting a defect, the same test should be repeated and added to the regression suite. Performance tests should also be included in the CI/CD pipeline so that every major release is checked before deployment. Additional improvements include database indexing, caching, asynchronous processing, load balancing, auto-scaling, queue management, capacity planning, continuous monitoring, alerting, and regular backup and recovery tests. These practices help ensure that the learning platform remains available, responsive, and reliable even during periods of heavy usage.

14. A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.

Testers should be involved throughout the Software Development Life Cycle (SDLC), not only after coding is completed. During the requirements phase, testers review requirements to check whether they are clear, complete, consistent, and testable. They can identify missing business rules, ambiguous statements, and incorrect assumptions before development begins. During the design phase, testers review workflows, database structures, APIs, security controls, and error-handling plans. While developers are coding, testers can prepare test cases, test data, automation scripts, and integration scenarios. This approach is known as shift-left testing, because testing activities begin earlier in the development process.

Early tester involvement improves quality because defects found in requirements or design are cheaper and easier to correct than defects found after release. It also prevents misunderstandings between customers, developers, and testers. For example, suppose an online banking requirement states that a user can transfer money after entering an OTP, but it does not explain what should happen when the OTP expires or the network disconnects during the transfer. A tester involved during requirements analysis can raise these questions early. The team can then define proper rules, such as displaying a clear message, preventing duplicate transactions, and allowing the user to check the transfer status. This prevents serious defects from appearing during system testing or in production.

A practical example is an e-commerce checkout process. If testers participate only after development, they may discover late that the system applies a discount incorrectly when a coupon expires, payment fails, or the cart contains restricted products. If they participate from the beginning, they can review the requirements, create decision tables, identify boundary conditions, design API tests, and prepare automation for important checkout paths. Developers can also create unit tests based on these conditions, reducing the number of defects that reach later testing stages.

Team collaboration should be improved through regular requirement reviews, three-amigos meetings involving a tester, developer, and business representative, shared acceptance criteria, and joint risk analysis. Testers and developers should communicate through tools such as Jira, Azure DevOps, or similar platforms, with clear defect reports containing reproduction steps, expected results, logs, and screenshots. Continuous integration should run unit, API, and regression tests whenever code changes are submitted. Code reviews, exploratory testing, root-cause analysis, and lessons-learned meetings should also be adopted. By making testers active contributors from requirements through maintenance, the team can detect defects earlier, reduce rework, improve communication, and deliver more reliable software.

15. A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.

A system that produces correct outputs in some situations but fails in others indicates incomplete test coverage and insufficient testing of different input conditions, business rules, system states, and user scenarios. The testing team may have focused only on normal or valid inputs and ignored invalid data, boundary values, exceptional conditions, different user roles, integration points, and performance-related situations. For example, if an application accepts marks from 0 to 100, testing only values such as 50 and 80 may not reveal errors for 0, 100, negative values, or values above 100. Proper test coverage ensures that important requirements, code paths, conditions, interfaces, and possible user actions are tested. Although complete testing is impossible, risk-based coverage helps prioritize critical functions and reduces the chance of defects escaping into production.

Test cases should be designed from clearly understood requirements and should include a test objective, preconditions, input data, execution steps, expected result, and pass/fail criteria. Testers should use equivalence partitioning to divide inputs into valid and invalid groups, boundary-value analysis to test values at and around limits, decision-table testing to verify combinations of business rules, and state-transition testing to check behavior as the system moves between states. Negative testing should confirm that the system rejects incorrect data with suitable messages, while exploratory testing can reveal unexpected defects that predefined cases may miss. Requirements should be linked to test cases through a traceability matrix so that every important requirement is covered. Automated unit, integration, and regression tests should be executed regularly, and test results should be compared with an independent expected-output source where possible. Defects should be analyzed for root cause, and every confirmed defect should lead to a regression test. These practices provide broader validation, improve reliability, and help ensure that the system behaves correctly across normal, exceptional, and boundary conditions.

16. A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.

A logistics application that experiences delays in defect identification usually suffers from unclear, incomplete, or inconsistent defect reports. A defect should be clearly defined as a deviation between the expected result and the actual result. Every report should explain what went wrong, where it occurred, under which conditions it appeared, and how seriously it affects the business. For example, instead of reporting delivery status is not working, the tester should state that when a shipment is marked as Delivered through the mobile application, the web portal continues to display In Transit after 15 minutes. Clear definitions help the team understand the problem quickly, reproduce it, assign it to the correct developer, and track it until resolution.

A structured defect report should include a unique defect ID, concise title, affected module, build or version number, environment, preconditions, test data, detailed reproduction steps, expected result, actual result, severity, priority, screenshots, error logs, and related test-case information. In a logistics system, additional details such as shipment number, warehouse location, delivery status, timestamp, device, network condition, and API response may be important. Such reports improve communication because testers, developers, business analysts, and managers receive the same factual information. They also reduce unnecessary clarification meetings and make it easier to distinguish duplicate defects from new issues.

The organization should adopt a common defect-reporting template and define standards for severity and priority. For example, a failure that loses shipment information should be classified as critical, while a minor spelling mistake may be classified as low severity. All defects should be recorded in a tracking tool such as Jira, Azure DevOps, or Bugzilla and moved through standard statuses such as New, Assigned, In Progress, Fixed, Retest, Reopened, and Closed. Teams should conduct regular defect-triage meetings, attach logs automatically where possible, link defects to requirements and test cases, and perform root-cause analysis for repeated problems. Report quality should also be reviewed, and testers should receive training in writing precise, objective, and reproducible reports. These improvements make defect handling faster, improve collaboration, and help prevent similar failures in future releases.

17. A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.

A software team that struggles to maintain consistency in testing across modules should adopt a process-based testing approach, where testing activities follow a defined, repeatable, and measurable process. The process should begin with reviewing requirements, identifying risks, defining test conditions, preparing test cases, executing them, recording defects, and performing regression testing after fixes. A common testing life cycle, standard templates, shared terminology, and clear entry and exit criteria should be used by all module teams. Requirements should be linked to test cases and defects through a traceability matrix so that every important function is tested consistently.

Standardized processes improve quality by reducing differences in the way testers plan, execute, document, and report tests. They ensure that testers do not depend only on personal experience or informal methods. For example, all modules should define expected results, test valid and invalid inputs, verify boundary values, test error handling, and confirm that data is correctly transferred between connected components. Common severity and priority definitions also ensure that defects are evaluated fairly across modules.

Consider an online shopping system with separate login, product, payment, and order modules. If the payment team tests only successful transactions while the order team tests failed payments and duplicate submissions, inconsistent defects may remain undetected. A process-based approach would require every module to use a common checklist covering positive, negative, boundary, security, integration, and recovery scenarios. The payment module would then be tested not only for successful payment but also for declined cards, network interruption, timeout, duplicate clicks, and payment success with delayed order confirmation. The same standards would be applied to the other modules.

Consistency can be improved through shared test-case templates, common coding and automation standards, centralized test repositories, peer reviews, risk-based test planning, and regular cross-module test reviews. Automated smoke and regression suites should run through the CI/CD pipeline, while dashboards can track coverage, defect leakage, failed tests, and reopened defects. Teams should also conduct training, maintain reusable test data, hold regular coordination meetings, and perform root-cause analysis for repeated failures. These practices create a controlled testing process, improve communication, reduce missed scenarios, and produce more reliable software across all modules.

18. A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.

A system that fails after deployment because of untested edge cases shows that the testing process focused mainly on normal conditions and did not sufficiently examine unusual, extreme, or invalid situations. Testers may have checked common inputs and successful workflows but ignored maximum and minimum values, empty fields, special characters, unexpected user actions, network interruptions, expired sessions, large data volumes, and unusual combinations of conditions. These cases are often missed because of incomplete requirements, limited test data, time pressure, poor risk analysis, or repeatedly executing the same routine test cases. Applying testing principles requires the team to recognize that exhaustive testing is impossible and therefore prioritize high-risk functions, business-critical boundaries, and situations that may cause serious failure after release.

Boundary-value testing is especially useful because defects frequently occur at the edges of valid input ranges rather than in the middle. For example, if a logistics system allows a package weight between 1 and 50 kilograms, testers should check values such as 0, 1, 2, 49, 50, and 51. If a password must contain 8 to 20 characters, tests should include 7, 8, 9, 19, 20, and 21 characters. Similar testing should be applied to dates, file sizes, transaction limits, user permissions, and database capacity. Edge cases can be included by reviewing requirements and identifying every limit, exception, dependency, and possible failure condition. Testers should use equivalence partitioning, boundary-value analysis, decision tables, state-transition testing, negative testing, and exploratory testing. Test data should include null values, duplicate records, incorrect formats, extreme numbers, special characters, simultaneous requests, interrupted connections, and partially completed transactions. A traceability matrix can confirm that requirements and risks are covered, while automated regression tests should be added for every discovered edge-case defect. Improvements such as early tester involvement, risk-based planning, peer review of test cases, realistic test data, production-like environments, continuous integration, and root-cause analysis will help identify hidden conditions before deployment. This improves reliability by ensuring that the system is validated not only for expected usage but also for unusual conditions that real users may encounter.

- 19. A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.**

A company that lacks proper defect documentation will find it difficult to track, prioritize, resolve, and prevent software issues. To solve this problem, the team should create a centralized defect repository, which is a structured system containing complete information about every defect found during testing, production, or customer use. Each defect should have a unique identification number, clear title, affected module, build or version, environment, severity, priority, date reported, reporter, assigned developer, current status, reproduction steps, expected result, actual result, screenshots, logs, and related test case. For example, instead of recording login problem, the repository should state that users are unable to log in when the password contains a special character in version 3.2 on Android 14. Such detailed documentation allows developers to reproduce the issue quickly and helps testers verify whether the correction is successful.

Documentation improves defect management by providing a complete history of the issue from discovery to closure. Defects can be assigned, monitored, retested, reopened, or closed using standard statuses such as New, Assigned, In Progress, Fixed, Retest, Reopened, and Closed. A structured repository also supports analysis by storing information that can be grouped according to module, defect type, severity, root cause, release, and responsible component. For example, if many defects are repeatedly reported in the payment module, the team can identify it as a high-risk area and increase integration, security, and negative testing there. Similarly, repeated defects caused by incorrect requirements may show the need for better requirement reviews. The company should use tools such as Jira, Azure DevOps, Bugzilla, or an equivalent tracking system, adopt a common defect-reporting template, define clear severity and priority standards, link defects with requirements and test cases, and conduct regular defect-triage meetings. Testers should attach logs, screenshots, and videos where necessary, avoid unclear language, and perform root-cause analysis for recurring problems. Every fixed defect should also produce a regression test. These practices create reliable historical data, improve communication between testers and developers, reduce duplicate reports, speed up resolution, and help prevent similar defects in future releases.

20. A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.

A banking system that repeatedly fails during transaction processing indicates weaknesses in the existing testing approach. The team may be testing only successful transactions and normal conditions while ignoring invalid inputs, network failures, duplicate requests, authorization problems, database errors, and transaction limits. To redesign the approach, testing should begin with a clear review of requirements and risk analysis. Critical functions such as login, account validation, balance checking, beneficiary verification, fund transfer, transaction confirmation, rollback, and notification should receive higher testing priority because their failure can cause financial loss or data inconsistency.

The testing process should follow a structured life cycle. Testers should prepare test conditions, test cases, test data, expected results, and traceability links before execution. Unit testing should verify individual calculation and validation functions, while integration testing should check communication between the mobile application, banking APIs, payment gateways, and databases. System testing should validate complete transfer workflows, and regression testing should confirm that new changes do not damage existing functions. Security testing, performance testing, usability testing, and recovery testing are also essential because banking applications handle sensitive information and require high reliability.

Structured testing improves quality by making testing repeatable, measurable, and complete. For example, a fund-transfer test should not check only whether money is successfully transferred. It should also verify insufficient balance, invalid account numbers, incorrect OTPs, expired sessions, exceeded transaction limits, service timeouts, loss of network after submission, and repeated clicks on the transfer button. If the network fails after the account is debited, the system must not debit the customer twice. It should either complete the transaction safely, reverse the debit, or display a clear pending status that can be checked later.

To improve defect detection, testers should use equivalence partitioning, boundary-value analysis, decision-table testing, state-transition testing, negative testing, exploratory testing, and risk-based testing. Boundary values such as zero balance, maximum transfer amount, minimum OTP length, and expired OTP time should be tested. Decision tables can verify combinations of account status, balance, beneficiary approval, transaction limit, and authentication result. State-transition testing can confirm correct movement from Initiated to Processing, Successful, Failed, Reversed, or Pending states. Automated API, unit, smoke, and regression tests should run through the CI/CD pipeline. Test environments should use realistic but protected data, and logs, transaction IDs, API responses, and database records should be monitored. Defects should be documented clearly, analyzed for root cause, and converted into permanent regression tests. These practices help the banking system detect failures earlier, prevent duplicate or incorrect transactions, and deliver safer and more dependable services.